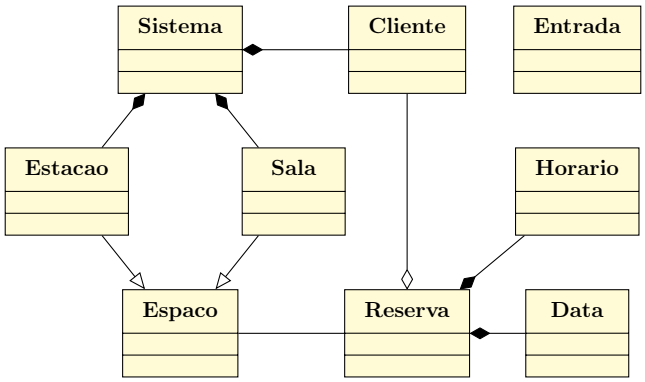
# Atividade 2: Implementação

POO - BSI - Ifes Serra

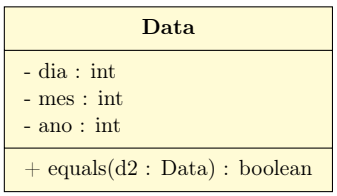
29 de abril de 2026

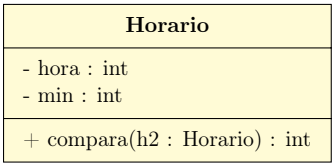
## 1 Diagrama de Classes UML

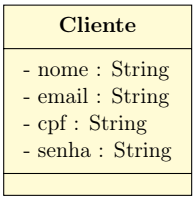
Nesta atividade, você deve começar a implementar o sistema feito na Atividade 1. Como acontece em vários projetos, a etapa de implementação demandou que alguns ajustes fossem feitos no diagrama de classes. O diagrama ainda pode vir a mudar na próxima etapa (a fase final), em que serão adicionados os conceitos de tratamento de erros e exceções, persistência em arquivos, classes abstratas, interfaces, coleções e ordenação. Por enquanto, as classes e relacionamentos a serem implementados estão mostrados no diagrama a seguir:




Note que há uma nova classe chamada Entrada, que vai cuidar de **toda a interação** com o usuário do sistema. **Nenhuma leitura de dados ou impressão pode ser feita fora da classe Entrada**. A seguir, uma descrição detalhada dos atributos e métodos que devem ser implementados em cada classe por enquanto:




O método **equals** na classe **Data** retorna um booleano indicando se a Data é a mesma que uma outra data recebida como parâmetro. Já o método **compara** na classe **Horario** recebe um outro horário **h2** retorna:

- -1, se o horário for anterior a h2;
- 1, se o horário for posterior a h2;
- 0, se os horários forem iguais.

As classes **Data**, **Horario**, **Cliente**, **Espaco**, **Estacao**, **Sala** e **Reserva** possuem um método **toString**. O método **toString** em uma classe Java qualquer indica como um objeto desta classe será representado quando for impresso. Para um objeto **d** da classe **Data** correspondente a 14 de fevereiro de 2025, por exemplo, o método deve retornar a String "14/02/2025". Com isso, o código a seguir:

```
System.out.println(d);
```

irá exibir na tela:

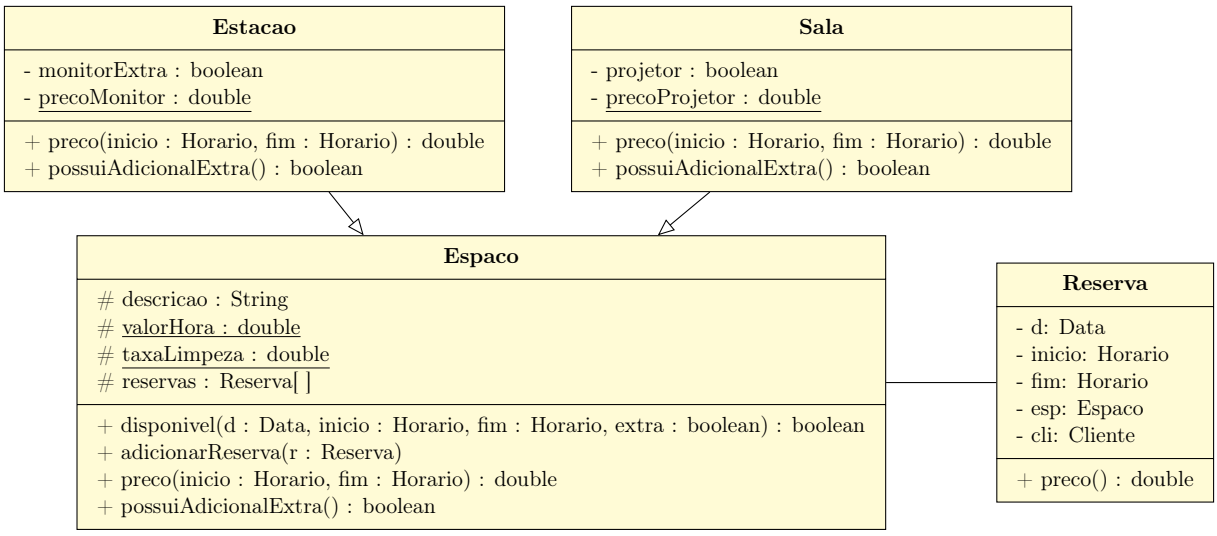
```
14/02/2025
```

É preciso pesquisar como formatar o número inteiro para que ele retorne uma String contendo exatamente dois algarismos. Analogamente, o método **toString** da classe **Horario** deve retorna uma String no seguinte formato:

```
16:20
```

Na classe **Cliente**, o método **toString** deverá exibir nome, cpf e email do cliente, no formato a seguir:

```
Pedro da Silva (pedro@silva.com - CPF: 123.456.789-00)
```




Os construtores e os métodos getters e setters serão omitidos de todos os diagramas. Crie os que forem necessários. O método **preco** da classe **Espaco** calcula o preco por hora de um determinado espaço:

$$(valorPorHora \times quantidadeDeHoras) + taxaDeLimpeza$$

Note que o agendamento é feito em horas inteiras. Uma reserva de 8h00 às 9h00 custa o mesmo que uma reserva de 8h20 às 8h50 (o equivalente a uma reserva de 1 hora inteira). Da mesma forma, uma reserva de 8h20 às 11h40 custa o equivalente a uma reserva de 4 horas.

O método **adicionarReserva** recebe uma reserva e adiciona-a à lista de reservas deste espaço. O método **possuiAdicionalExtra**, por enquanto, retorna sempre **true**. Ele será reescrito nas duas subclasses (**Sala** e **Estacao**). Já o método **disponivel** verifica se o espaço está disponível em um dado intervalo de tempo. Além do intervalo de tmepo, ele recebe um booleano indicando se o usuário deseja o item extra (monitor, no caso de uma estação de

trabalho, e projetor no caso de uma sala). Ele só deve retornar verdadeiro se o item extra presente neste espaço estiver de acordo com o requisito informado pelo usuário.

O método **preco** em uma estação de trabalho adiciona ao preço do espaço o valor extra caso a estação possua monitor extra. O método **possuiAdicionalExtra** reescreve o método da superclasse indicando se o monitor extra está disponível.

Já o preço de uma sala é 4 vezes o preço por hora de um espaço comum (pois as salas ocupam mais espaço físico que uma estação de trabalho). Além disso, também deve ser adicionado ao preço final o valor do projetor, caso ele possua. O método **possuiAdicionalExtra** também reescreve o método da superclasse indicando se o projetor está disponível.

Finalmente, o método **preco** na classe **Reserva** chama o cálculo do preço no espaço requisitado nesta reserva, de acordo com o horário inicial e final da reserva.

O método **toString** da classe **Espaco** retorna apenas sua descrição. Ex:

```
Sala Azul
```

O método nas classes **Estacao** e **Sala** reescrevem o método da superclasse, informando se o espaço é uma sala ou estação de trabalho e se possui item adicional ou não. Exemplos:

```
Estacao 01 (Estacao de Trabalho com Monitor Extra)
Estacao 02 (Estacao de Trabalho sem Monitor Extra)
Sala Azul (Sala com Projetor)
Sala Amarela (Sala sem Projetor)
```

Por fim, o método **toString** da classe **Reserva** retorna as informações da reserva no seguinte formato:

```
Reserva:
 * Local: Sala Azul (Sala com Projetor)
 * Data: 14/02/2025
 * Horario: 16:20
 * Cliente: Pedro da Silva (pedro@silva.com - CPF: 123.456.789-00)
 * Valor: R$ 110.00
```

Note que as informações sobre o local, data, horário e cliente na reserva utilizam os métodos **toString** das classes **Sala**/**Estacao**, **Data**, **Horario** e **Cliente**.

| **Sistema** |
|---|
| - salas : Espaco[ ]<br>- estacoes : Espaco[ ]<br>- clientes : Cliente[ ] |
| + getCliente(cpf : String) : Cliente<br>+ cadastrar(c : Cliente)<br>+ cadastrar(s : Sala)<br>+ cadastrar(e : Estacao)<br>+ reservar(tipo : String, d : Data, inicio : Horario, fim : Horario, c : Cliente, extra : boolean) : boolean<br>+ reservar(tipo : String, d : Data, c : Cliente, extra : boolean) : boolean<br>+ reservar(tipo : String, d : Data, turno : String, c : Cliente, extra : boolean) : boolean<br>+ getReservas() : Reserva[ ]<br>+ getReservas(d : Data) : Reserva[ ]<br>+ getReservas(c : Cliente) : Reserva[ ] |

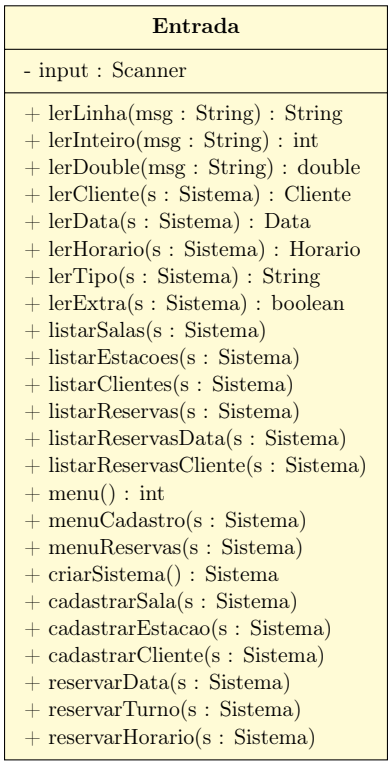
O método **getCliente** da classe **Sistema** recebe um cpf e retorna o **Cliente** com aquele cpf (null, caso o cliente não esteja cadastrado). Se o método **cadastrar** receber um **Cliente**, adiciona um cliente à lista de clientes. Se receber uma **Estacao**, adiciona-a à lista de estações

de trabalho. Da mesma forma, se receber uma **Sala**, adiciona esta sala à lista com todas as salas da empresa.

O método **reservar** verifica se há um espaço disponível para a data e horário informados que atenda ao requisito de item extra (projetor ou monitor). O método se comporta diferente dependendo dos parâmetros recebidos:

- Se receber uma data, deve reservar um espaço disponível para o dia inteiro (de 8h às 22h).
- Se receber uma data e dois horários, deve reservá-lo no intervalo de tempo indicado.
- Se receber uma data e um turno, deve reservar o espaço naquele turno. Se o turno for "matutino", a reserva será de 8h às 12h. Se for "vespertino", será de 13h às 17h. Se for "noturno", será de 18h às 22h.

O método **getReservas** sem parâmetro retorna todas as reservas do sistema (tanto de salas quanto de estações de trabalho). Se receber uma **Data**, filtra apenas as reservas naquela data. Se receber um **Cliente**, filtra apenas as reservas deste cliente.




A classe **Entrada** é a que possui mais métodos, incluindo toda a interação com o usuário. Alguns métodos já foram implementados no arquivo *Entrada.java* disponibilizado. Os métodos descritos para esta classe são **sugestões**. O importante é que todas as funcionalidades do menu sejam implementadas nesta classe.

Quando o sistema é iniciado, o método **criarSistema** lê as informações básicas do sistema. Ex:

```
Digite o valor por hora para usar um espaço: R$ 5
Digite a taxa de limpeza: R$ 10
Digite o valor extra para usar o projetor: R$ 10
Digite o valor para usar o monitor extra: R$ 5
```

Há um método com o *menu* do sistema:

```
Escolha uma opção:
1) Cadastros
2) Reservas
0) Sair
```

As opções de cadastro são:

```
Escolha uma opção:
1) Ver clientes
2) Ver salas
3) Ver estações de trabalho
4) Cadastrar cliente
5) Cadastrar sala
6) Cadastrar estação de trabalho
0) Voltar
```

As opções de reserva são:

```
Escolha uma opção:
1) Ver reservas
2) Ver reservas por data
3) Ver reservas por cliente
4) Fazer reserva (dia inteiro)
5) Fazer reserva (turno inteiro)
6) Fazer reserva (horário específico)
0) Voltar
```

A opção **Cadastrar cliente** lê os dados de um novo cliente, **nesta ordem**:

```
Digite o nome do cliente: Joao
Digite o CPF do cliente: 123
Digite o email do cliente: joao@joao.com
Digite a senha do cliente: 123
```

A opção **Cadastrar sala** lê os dados de uma nova sala, **nesta ordem**:

```
Digite o nome da sala: Sala Azul
Possui projetor? (s/n): s
```

A opção **Cadastrar estação de trabalho** lê os dados de uma nova estação, **nesta ordem**:

```
Digite o nome da estação de trabalho: Estacao 01
Possui monitor extra? (s/n): s
```

A opção **Ver clientes** exibe todos os clientes cadastrados (se houver):

```
Clientes cadastrados:
Joao (joao@joao.com - CPF: 123)
Maria (maria@maria.com - CPF: 234)
Pedro (pedro@pedro.com - CPF: 345)
```

A opção **Ver salas** exibe todos as salas cadastradas (se houver):

```
Salas cadastradas:
Sala Azul (Sala com Projetor)
Sala Amarela (Sala sem Projetor)
Sala Verde (Sala sem Projetor)
```

A opção **Ver estações de trabalho** exibe todos as estações de trabalho cadastradas (se houver):

```
Estações de Trabalho cadastradas:
Estacao 01 (Estacao de Trabalho com Monitor Extra)
Estacao 02 (Estacao de Trabalho sem Monitor Extra)
Estacao 03 (Estacao de Trabalho com Monitor Extra)
```

A opção **Fazer reserva (dia inteiro)** lê os dados de uma nova reserva para um dia inteiro, **nesta ordem**:

```
Deseja reservar uma sala ou estação de trabalho? (s/e): s
Deseja reservar sala com projetor? (s/n): s
Escolha uma data (dd/mm/aaaa):
Dia: 2
Mês: 2
Ano: 2026
********************************
Clientes cadastrados:
Joao (joao@joao.com - CPF: 123)
Maria (maria@maria.com - CPF: 234)
Pedro (pedro@pedro.com - CPF: 345)
Digite o CPF do cliente: 123
Reserva realizada com sucesso!
```

A opção **Fazer reserva (turno inteiro)** lê os dados de uma nova reserva para um turno inteiro, **nesta ordem**:

```
Deseja reservar uma sala ou estação de trabalho? (s/e):s
Deseja reservar sala com projetor? (s/n): s
Escolha uma data (dd/mm/aaaa):
Dia: 2
Mês: 2
Ano: 2026
Escolha um turno: matutino, vespertino ou noturno (m/v/n): v
********************************
Clientes cadastrados:
Joao (joao@joao.com - CPF: 123)
Maria (maria@maria.com - CPF: 234)
Pedro (pedro@pedro.com - CPF: 345)
Digite o CPF do cliente: 234
Reserva não realizada.
```

A opção **Fazer reserva (horário específico)** lê os dados de uma nova reserva para um horário específico, **nesta ordem**:

```
Deseja reservar uma sala ou estação de trabalho? (s/e):s
Deseja reservar sala com projetor? (s/n): n
Escolha uma data (dd/mm/aaaa):
Dia: 2
Mês: 2
Ano: 2026
Escolha um horário (hh:mm):
Hora: 16
Minuto: 0
Escolha um horário (hh:mm):
Hora: 20
Minuto: 0
********************************
Clientes cadastrados:
Joao (joao@joao.com - CPF: 123)
Maria (maria@maria.com - CPF: 234)
Pedro (pedro@pedro.com - CPF: 345)
Digite o CPF do cliente: 345
Reserva realizada com sucesso!
```

As reservas para estações de trabalho seguem o mesmo fluxo de reservas para salas, indicando se há monitor extra no lugar de projetor.

A opção de **Ver reservas** mostra todas as reservas do sistema:

```
Reservas cadastradas:
Reserva:
 * Local: Sala Azul (Sala com Projetor)
 * Data: 02/02/2026
 * Horario: 08:00 - 22:00
 * Cliente: Joao (joao@joao.com - CPF: 123)
 * Valor: R$ 330,00

Reserva:
 * Local: Sala Azul (Sala com Projetor)
 * Data: 03/02/2026
 * Horario: 08:00 - 10:00
 * Cliente: Pedro (pedro@pedro.com - CPF: 345)
 * Valor: R$ 90,00

Reserva:
 * Local: Sala Amarela (Sala sem Projetor)
 * Data: 02/02/2026
 * Horario: 13:00 - 17:00
 * Cliente: Maria (maria@maria.com - CPF: 234)
 * Valor: R$ 120,00

Reserva:
 * Local: Sala Verde (Sala sem Projetor)
 * Data: 02/02/2026
 * Horario: 16:00 - 20:00
 * Cliente: Pedro (pedro@pedro.com - CPF: 345)
```

```
 * Valor: R$ 120,00

Reserva:
 * Local: Estacao 02 (Estacao de Trabalho sem Monitor Extra)
 * Data: 02/02/2026
 * Horario: 08:00 - 22:00
 * Cliente: Joao (joao@joao.com - CPF: 123)
 * Valor: R$ 80,00
```

A opção de **Ver reservas por data** lê o dia, mês e ano (um de cada vez) e depois mostra todas as reservas do sistema para aquela data:

```
Escolha uma data (dd/mm/aaaa):
Dia: 2
Mês: 2
Ano: 2026
Reservas cadastradas nesta data:
Reserva:
 * Local: Sala Azul (Sala com Projetor)
 * Data: 02/02/2026
 * Horario: 08:00 - 22:00
 * Cliente: Joao (joao@joao.com - CPF: 123)
 * Valor: R$ 330,00

Reserva:
 * Local: Sala Amarela (Sala sem Projetor)
 * Data: 02/02/2026
 * Horario: 13:00 - 17:00
 * Cliente: Maria (maria@maria.com - CPF: 234)
 * Valor: R$ 120,00

Reserva:
 * Local: Sala Verde (Sala sem Projetor)
 * Data: 02/02/2026
 * Horario: 16:00 - 20:00
 * Cliente: Pedro (pedro@pedro.com - CPF: 345)
 * Valor: R$ 120,00

Reserva:
 * Local: Estacao 02 (Estacao de Trabalho sem Monitor Extra)
 * Data: 02/02/2026
 * Horario: 08:00 - 22:00
 * Cliente: Joao (joao@joao.com - CPF: 123)
 * Valor: R$ 80,00
```

A opção de **Ver reservas por cliente** lê o cpf de um cliente e mostra todas as reservas do sistema para este cliente:

```
Clientes cadastrados:
```

```
Joao (joao@joao.com - CPF: 123)
Maria (maria@maria.com - CPF: 234)
Pedro (pedro@pedro.com - CPF: 345)
Digite o CPF do cliente: 345
Reservas cadastradas para este cliente:
Reserva:
 * Local: Sala Azul (Sala com Projetor)
 * Data: 03/02/2026
 * Horario: 08:00 - 10:00
 * Cliente: Pedro (pedro@pedro.com - CPF: 345)
 * Valor: R$ 90,00

Reserva:
 * Local: Sala Verde (Sala sem Projetor)
 * Data: 02/02/2026
 * Horario: 16:00 - 20:00
 * Cliente: Pedro (pedro@pedro.com - CPF: 345)
 * Valor: R$ 120,00
```

Se necessário, crie métodos **getters** para acessar o valor de algum atributo que esteja encapsulado nas classes. Para cada conjunto de objetos no diagrama, o aluno deve decidir se é mais vantajoso implementá-lo usando array ou ArrayList. Apenas estas duas estruturas são permitidas nesta atividade.

É importante ler os dados de cadastros e opções do menu **na ordem em que foram descritos neste documento**, pois a correção será automatizada. Na página da disciplina, há um arquivo "input.txt" com um exemplo de entrada do arquivo. Basta rodar seu programa com esse arquivo na pasta onde o programa está sendo executado, e o arquivo será lido automaticamente (ao invés do programa ler a entrada do teclado). Faça isso apenas quando todas as opções do seu menu já estiverem funcionando.

# 2 Observações

1. A atividade vale 40 pontos e deve ser entregue até 12/05.
2. O código deve ser feito em Java, considerando as boas práticas de POO (como reuso de código, por exemplo, sempre que possível).
3. A atividade pode ser feita em grupos de até dois integrantes.
4. Atividades entregues após o prazo serão automaticamente rejeitados.
5. Atividades com erro de execução, com formato de saída incorreto, ou que não compilarem terão nota 0.
6. Atividades considerados plágio terão nota 0 para quem copiou e para quem forneceu o exercício, e serão enviados ao Conselho de Ética.
7. A Atividade deve ser enviado na sala da disciplina do AVA.
8. Em caso de dúvidas na especificação da atividade ou na própria atividade, contate-me em sala de aula.